repo-to-pdf

Contents

src/bin/index.js

to top

```
#!/usr/bin/env node
const program = require('commander')

const { getSizeInByte } = require('../utils')
const { generateEbook } = require('../html')

const version = require('../../package.json').version

let inputFolder, outputFile
program
  .version('repo-to-pdf ' + version)
  .usage('<input> [output] [options]')
  .arguments('<input> [output] [options]')
  .option('-d, --device <platform>', 'device [desktop(default)|mobile|tablet]',
'desktop')
  .option('-t, --title [name]', 'title')
  .option('-w, --whitelist [wlist]', 'file format white list, split by ,')
  .option('-s, --size [size]', 'pdf file size limit, in Mb')
  .option('-r, --renderer <engine>', '[node|wkhtmltopdf|calibre] use
node(puppeteer), wkhtmltopdf or calibre to render pdf', 'node')
  .option('-f, --format <ext>', 'output format, pdf|mobi|epub', 'pdf')
  .option('--no-outline', 'disable PDF outlines/bookmarks (default enabled)')
  .option('-p, --concurrency <num>', 'number of parallel Puppeteer PDF render
jobs')
  .option('-c, --calibre [path]', 'path to calibre')
  .option('-b, --baseUrl [url]', 'base url of html folder. By default file:// is
used.')
  .action(function(input, output) {
    inputFolder = input
    outputFile = output
  })

program.parse(process.argv)

const PDF_SIZE    = getSizeInByte(10) // 10 Mb
const title       = program.title || inputFolder
const device      = program.device
const pdf_size    = program.size ? getSizeInByte(program.size) : PDF_SIZE
const format      = program.format
const white_list  = program.whitelist
const renderer    = program.renderer
const calibrePath = program.calibre
const baseUrl     = program.baseUrl
const outline     = program.outline
const concurrency = program.concurrency

generateEbook(inputFolder, outputFile, title, {
  renderer,
  calibrePath,
  pdf_size,
```

```
  white_list,
  format,
  device,
  baseUrl,
  outline,
  concurrency,
}).catch((error) => {
  console.error(error)
  process.exitCode = 1
})
```

src/html.js

to top

```
const path = require('path')
const fs = require('fs')
const os = require('os')

const { Remarkable } = require('remarkable')
const hljs = require('highlight.js')

const RepoBook = require('./repo')
const { sequenceRenderEbook, sequenceRenderPDF, renderPDF } =
require('./render')
const { getFileName, getFileNameExt } = require('./utils')

function getRemarkableParser() {
  return new Remarkable({
    breaks: true,
    highlight: function(str, lang) {
      if (lang && hljs.getLanguage(lang)) {
        try {
          return hljs.highlight(lang, str).value
        } catch (err) {}
      }

      try {
        return hljs.highlightAuto(str).value
      } catch (err) {}

      return ''
    },
  }).use(function(remarkable) {
    remarkable.renderer.rules.heading_open = function(tokens, idx) {
      return '<h' + tokens[idx].hLevel + ' id=' + tokens[idx + 1].content + '
anchor=true>'
    }
  })
}

// => './path/file-1.html'
function getHTMLFiles(mdString, repoBook, options) {
  const { pdf_size, white_list, device, baseUrl, protocol, renderer,
outputFileName, inputFolder } = options
  const opts = {
    cssPath: {
      desktop: '/css/github-min.css',
      tablet: '/css/github-min-tablet.css',
      mobile: '/css/github-min-mobile.css',
    },
    highlightCssPath: '/css/vs.css',
    relaxedCSS: {
      desktop: '',
      tablet: `@page {
```

```
                size: 8in 14in;
                -relaxed-page-width: 8in;
                -relaxed-page-height: 14in;
                margin: 0;
              }`,
      mobile: `@page {
                size: 6in 10in;
                -relaxed-page-width: 6in;
                -relaxed-page-height: 10in;
                margin: 0;
              }`,
    },
  }
  const HTMLFileNameWithExt = getFileNameExt(outputFileName, 'html') ||
getFileName(inputFolder) + '.html'
  let outputFile = path.resolve(process.cwd(), HTMLFileNameWithExt)

  const mdParser = getRemarkableParser()

  const mdHtml = `<article class="markdown-body">` + mdParser.render(mdString) +
`</article>`
  const indexHtmlPath = path.join(__dirname, '../html5bp', 'index.html')
  const htmlString = fs
    .readFileSync(indexHtmlPath, 'utf-8')
    // TODO: this sits before content replacing, to prevent replacing baseUrl in
content text
    .replace(/\{\{baseUrl\}\}/g, protocol + baseUrl)
    .replace('{{cssPath}}', protocol + baseUrl + opts.cssPath[device])
    .replace('{{highlightPath}}', protocol + baseUrl + opts.highlightCssPath)
    .replace('{{relaxedCSS}}', opts.relaxedCSS[device])
    .replace('{{content}}', mdHtml)

  if (!repoBook.hasSingleFile()) {
    outputFile = outputFile.replace('.html', '-' + repoBook.currentPart() +
'.html')
  }
  fs.writeFileSync(outputFile, htmlString)
  return outputFile
}

function checkOptions(options) {
  if (options.outline === undefined) {
    options.outline = true
  }

  if (options.concurrency !== undefined) {
    const concurrency = Number(options.concurrency)
    if (!Number.isInteger(concurrency) || concurrency < 1) {
      console.log(`Invalid concurrency "${options.concurrency}", fallback to
auto mode.`)
      delete options.concurrency
```

```
    } else {
      options.concurrency = concurrency
    }
  }

  if (options.baseUrl) {
    options.protocol = ''
  } else {
    options.protocol = os.name === 'windows' ? 'file:///' : 'file://'
    options.baseUrl = path.resolve(__dirname, '../html5bp')
  }

  if (options.format !== 'pdf' && options.renderer === 'node') {
    console.log(`Try to create ${options.format}, use renderer Calibre.`)
    options.renderer = 'calibre'
  }

  // check calibre path
  const calibrePaths = ['/Applications/calibre.app/Contents/MacOS/ebook-
convert', '/usr/bin/ebook-convert']
  if (options.calibrePath) {
    calibrePaths.unshift(options.calibrePath)
  }

  let i = 0
  for (; i < calibrePaths.length; i++) {
    if (fs.existsSync(calibrePaths[i])) {
      options.calibrePath = calibrePaths[i]
      break
    }
  }
  if (i === calibrePaths.length && ['mobi', 'epub'].includes(options.format)) {
    console.log('Calibre ebook-convert not found, make sure you specify the path
by --calibre /path/to/ebook-convert.')
    return false
  }
  return true
}

/**
 * @typedef Options
 * @type {object}
 * @property {string} renderer - [node|calibre|wkhtmltopdf] can be either node,
calibre and wkhtmltopdf
 * @property {string} calibrePath - path of calibre's ebook-convert
 * @property {string} pdf_size - pdf size limit, in bytes
 * @property {string} white_list - list of file extensions to be included,
separate by ','
 * @property {string} format - [mobi|epub|pdf] can be either mobi, epub, pdf
 * @property {string} device - [desktop|tablet|mobile] style can be opt for
desktop, tablet and mobile
```

```
 * @property {string} baseUrl - base url of CSS style files
 * @property {boolean} outline - include PDF outline/bookmarks (default true)
 * @property {number} concurrency - max number of parallel Puppeteer render jobs
(default auto)
 */

/**
 * Generate ebook from content folder with the given file format
 * @param {string} inputFolder - content folder
 * @param {string} outputFile - output file name
 * @param {string} title - title in ebook file
 * @param {Options} options
 */
async function generateEbook(inputFolder, outputFile, title, options = {
renderer: 'node' }) {
  if (!checkOptions(options)) {
    return
  }

  const { pdf_size, white_list, renderer } = options
  const repoBook = new RepoBook(inputFolder, title, pdf_size, white_list)

  const defaultOutputFileName = getFileName(inputFolder) + '.pdf'
  const outputFileName = outputFile || defaultOutputFileName

  options.outputFileName = outputFileName
  options.inputFolder = inputFolder
  options.outputFile = outputFile

  const outputFiles = []
  while (repoBook.hasNextPart()) {
    const mdString = repoBook.render()
    if (!mdString) {
      console.log('Nothing to generate.')
      return
    }
    let outputFile = null
    outputFile = getHTMLFiles(mdString, repoBook, options)

    if (!outputFile) {
      console.log('Fail to generate HTML files that are used to generate PDFs.')
      break
    }
    outputFiles.push(outputFile)
  }
  if (renderer === 'calibre') {
    sequenceRenderEbook(outputFiles, options)
  } else if (renderer === 'node') {
    await sequenceRenderPDF(outputFiles, options)
  } else if (renderer === 'wkhtmltopdf') {
    renderPDF(outputFiles, options)
```

```
  }
}

module.exports = { generateEbook }
```

src/index.js

to top

```
const { generateEbook } = require('./html')
module.exports = { generateEbook }
```

src/puppeteer.js

to top

```
const puppeteer = require('puppeteer');
const pageRenderTimeout = 10;

class HTML2PDF {
  constructor() {
    this.puppeteerConfig = {
      headless: 'new',
      args: [
        '--no-sandbox',
        '--disable-dev-shm-usage',
      ],
    };

    this._initialized = false;
    this._initializing = null;
    this.chrome = null;
  }

  async _initializePlugins() {
    if (this._initialized) {
      return;
    }

    if (this._initializing) {
      await this._initializing;
      return;
    }

    this._initializing = (async () => {
      this.chrome = await puppeteer.launch(this.puppeteerConfig);
      this._initialized = true;
    })();

    try {
      await this._initializing;
    } finally {
      this._initializing = null;
    }
  }

  async close() {
    if (this._initializing) {
      await this._initializing;
    }

    if (this.chrome) {
      await this.chrome.close();
    }
    this.chrome = null;
    this._initialized = false;
```

```
  }

  async pdf(templatePath, outputPath, options = {}) {
    await this._initializePlugins();
    const { outline = true } = options;
    const puppeteerPage = await this.chrome.newPage();
    await puppeteerPage.setJavaScriptEnabled(false);

    try {
      await puppeteerPage.goto('file:' + templatePath, {
        waitUntil: 'load',
        timeout: 1000 * (pageRenderTimeout || 30)
      })
      const pdfOptions = {
        path: outputPath,
        displayHeaderFooter: false,
        printBackground: true,
        timeout: 3 * 60 * 1000,
        outline,
      };

      await puppeteerPage.pdf(pdfOptions);
    } finally {
      await puppeteerPage.close();
    }
  }
}

module.exports = new HTML2PDF();
```

src/render.js

to top

```
const path = require('path')
const fs = require('fs')
const os = require('os')
const { spawnSync } = require('child_process')

const { getFileNameExt } = require('./utils')
const html2PDF = require('./puppeteer');
const wkhtml2PDF = require('./wkhtmltopdf');

let startTs

function removeRelaxedjsTempFiles(outputFileName) {
  const prefix =
outputFileName.split('.').reverse().slice(1).reverse().join('.')
  const html = spawnSync('rm', [`${prefix}.html`])
  const htm = spawnSync('rm', [`${prefix}_temp.htm`])
}

function reportPerformance(outputFileName, startTs) {
  const ts = (Date.now() - startTs) / 1000
  console.log(`${outputFileName} created in ${ts} seconds.`)
}

function getPDFConcurrency(totalFiles, options = {}) {
  if (totalFiles <= 1) {
    return 1
  }

  const requested = Number(options.concurrency)
  if (Number.isInteger(requested) && requested > 0) {
    return Math.min(totalFiles, requested)
  }

  const cpuCount = (os.cpus() || []).length || 1
  // Keep memory usage predictable while still using multiple cores.
  return Math.min(totalFiles, Math.max(1, Math.min(4, cpuCount)))
}

async function sequenceRenderPDF(docFiles, options = {}) {
  const concurrency = getPDFConcurrency(docFiles.length, options)

  try {
    if (concurrency === 1) {
      await docFiles.reduce(
        (promise, file) => promise.then(() => html2PDF.pdf(file,
file.replace('.html', '.pdf'), options)),
        Promise.resolve()
      )
      return
    }
```

```
    let nextFileIndex = 0
    const renderNext = async () => {
      const i = nextFileIndex
      nextFileIndex += 1
      if (i >= docFiles.length) {
        return
      }

      const file = docFiles[i]
      await html2PDF.pdf(file, file.replace('.html', '.pdf'), options)
      await renderNext()
    }

    const workers = Array.from({ length: concurrency }, () => renderNext())
    await Promise.all(workers)
  } finally {
    // close chrome so that cli can terminate
    await html2PDF.close()
  }
}

function renderPDF(docFiles, options = {}) {
  const parallel = 1;
  const total = docFiles.length;
  for (let i = 0; i < total;) {
    let j = 0;
    for (; j < parallel && (i + j) < total; j++) {
      const file = docFiles[i + j];
      wkhtml2PDF(file, file.replace('.html', '.pdf'), options);
    }
    i += j;
  }
}

function sequenceRenderEbook(docFiles, options, i = 0) {
  const { outputFileName, renderer, calibrePath, format } = options

  if (i === 0) {
    startTs = Date.now()
  }
  if (i >= docFiles.length) {
    if (renderer === 'node') {
      removeRelaxedjsTempFiles(outputFileName)
    }
    reportPerformance(outputFileName, startTs)
    return
  }
  const docFile = path.resolve(process.cwd(), docFiles[i])
  const formatFile = getFileNameExt(docFile, format)
```

```
  const formatArgs = {
    pdf: [
      '--pdf-add-toc',
      '--paper-size',
      'a4',
      '--pdf-default-font-size',
      '12',
      '--pdf-mono-font-size',
      '12',
      '--pdf-page-margin-left',
      '2',
      '--pdf-page-margin-right',
      '2',
      '--pdf-page-margin-top',
      '2',
      '--pdf-page-margin-bottom',
      '2',
      '--page-breaks-before',
      '/',
    ],
    mobi: ['--mobi-toc-at-start', '--output-profile', 'kindle_dx'],
    epub: ['--epub-inline-toc', '--output-profile', 'ipad3', '--flow-size',
'1000'],
  }

  const args = {
    calibre: [calibrePath, [docFile, formatFile].concat(formatArgs[format])]
  }

  if (renderer === 'calibre') {
    if (!fs.existsSync(calibrePath)) {
      console.log(`Calibre is not available.`)
    }
  }

  const cmd = args[renderer]
  const res = spawnSync(cmd[0], cmd[1])
  if (res.error) {
    console.log(`Some error happened when creating ${formatFile}.`)
    console.error(res.error);
    return
  }

  if (!fs.existsSync(formatFile)) {
    console.log(`${docFiles[i]} was not rendered. ${formatFile}`)
  }

  sequenceRenderEbook(docFiles, options, i + 1)
}
```

```
module.exports = { sequenceRenderEbook, sequenceRenderPDF, renderPDF }
```

src/repo.js

to top

```
const path = require('path')
const fs = require('fs')
const hljs = require('highlight.js')
const { getFileName, getCleanFilename } = require('./utils')

class RepoBook {
  constructor(dir, title, pdf_size, white_list) {
    this.title = title
    this.pdf_size = pdf_size

    this.blackList = ['node_modules', 'vendor']
    this.whiteList = white_list ? white_list.split(',') : null

    this.aliases = {}
    this.byteOffset = 0
    this.fileOffset = 0
    this.partOffset = 0
    this.done = false
    this.dir = dir

    this.files = this.readDir(dir)
    this.registerLanguages()
  }

  hasNextPart() {
    return this.done === false
  }

  hasSingleFile() {
    return this.partOffset === 0 // effective after at least calling render()
once
  }

  currentPart() {
    return this.partOffset
  }

  registerLanguage(name, language) {
    const lang = language(hljs)
    if (lang && lang.aliases) {
      name = name.split('.')[0]
      if (this.whiteList) {
        if (this.whiteList.indexOf(name) > -1) {
          this.aliases[name] = name
        }
      } else {
        this.aliases[name] = name
      }

      lang.aliases.map(alias => {
```

```
        if (this.whiteList) {
          if (this.whiteList.indexOf(alias) > -1) {
            this.aliases[alias] = name.split('.')[0]
          }
        } else {
          this.aliases[alias] = name.split('.')[0]
        }
        return null
      })
    }
  }

  registerLanguages() {
    const listPath = path.join(path.dirname(require.resolve('highlight.js')),
'languages')
    fs.readdirSync(listPath).map(f => {
      this.registerLanguage(f, require(path.join(listPath, f)))
      return null
    })
  }

  readDir(dir, allFiles = [], level = 0) {
    const files = fs
      .readdirSync(dir)
      .map(f => path.join(dir, f))
      .filter(f => fs.lstatSync(f).size / 1000 < 2000) // smaller than 2m
      .map(f => [f, level])

    level > 0 ? allFiles.push([dir, level, true]) : null // push folder name

    files.map(pair => {
      const f = pair[0]
      const blackListHit = this.blackList.filter(e => !!f.match(e)).length > 0
      if (!fs.lstatSync(f).isDirectory()) {
        allFiles.push([f, level + 1, false])
      } else {
        path.basename(f)[0] !== '.' && // ignore hidden folders
          blackListHit === false &&
          this.readDir(f, allFiles, level + 1)
      }
      return null
    })
    return allFiles
  }

  renderIndex(files) {
    return files
      .filter(f => {
        const fileName = getFileName(f[0])
        const ext = path.extname(fileName).slice(1)
        const isFolder = fs.lstatSync(f[0]).isDirectory()
```

```
        return fileName[0] !== '.' && (ext in this.aliases || isFolder)
      })
      .map(f => {
        const indexName = getCleanFilename(f[0], this.dir, f[1]),
          anchorName = getCleanFilename(f[0], this.dir),
          left_pad = f[2] ? '    '.repeat(f[1]) : '',
          h_level = '####',
          list_style = f[2] ? '' : '    '.repeat(f[1]) + '|-
'
        return f[2] ? `${h_level} ${left_pad} ${list_style} /${indexName}` :
`${h_level} ${left_pad}[${list_style} ${indexName}](#${anchorName})`
      })
      .join('\n')
  }

  render() {
    const files = this.files
    const contents = []
    let i = this.fileOffset

    for (; i < files.length; i++) {
      const file = files[i][0]

      const fileName = getFileName(file)
      if (fs.statSync(file).isDirectory()) {
        continue
      }

      const ext = path.extname(fileName).slice(1)

      if (ext.length === 0) {
        continue
      }

      if (fileName[0] === '.') {
        continue
      }

      const lang = this.aliases[ext]
      if (lang) {
        let data = fs.readFileSync(file)
        if (ext === 'md') {
          data = `#### ${getCleanFilename(file, this.dir)} \n[to top]
(#Contents)` + '\n' + data + '\n'
        } else {
          data = `#### ${getCleanFilename(file, this.dir)} \n[to top]
(#Contents)` + '\n``` ' + lang + '\n' + data + '\n```\n'
        }
        contents.push(data)

        this.byteOffset += data.length * 2
```

```
        if (this.byteOffset > this.pdf_size) {
          // if more than one part
          const title = `# ${this.title} (${++this.partOffset})\n\n\n\n`

          let toc = '## Contents\n'
          const index = this.renderIndex(files.slice(this.fileOffset, i + 1))
          toc += index
          contents.unshift(title, toc)

          if (i === this.fileOffset) {
            // when single file exceeds size limit
            this.fileOffset++
          } else {
            this.fileOffset = i
          }
          this.byteOffset = 0

          return contents.join('\n')
        }
      }
    }

    if (contents.length === 0) {
      this.done = true
      return null
    }

    // if one part
    const title = this.partOffset
      ? `# ${this.title} (${++this.partOffset})\n\n\n\n` // the last part
      : `# ${this.title} \n\n\n\n` // single pdf

    let toc = '## Contents\n'
    const index = this.renderIndex(files.slice(this.fileOffset, i + 1))
    toc += index
    contents.unshift(title, toc)

    this.fileOffset = files.length // should return null next round
    this.done = true

    return contents.join('\n')
  }
}

module.exports = RepoBook
```

src/utils.js

to top

```
const path = require('path')

function getSizeInByte(mb) {
  return mb * 0.8 * 1000 * 1000
}

// '../' => 'untitled'
// './'  => 'untitled'
// './path'  => 'path'
function getFileName(fpath) {
  const base = path.basename(fpath)
  return base[0] === '.' ? 'untitled' : base
}

function getCleanFilename(filename, folder, depth = 0) {
  filename = filename.replace(folder, '')
  if (folder === './') {
    depth -= 1
  }
  if (depth > 0) {
    return filename
      .split('/')
      .slice(depth)
      .join('/')
  } else {
    return filename
  }
}

// 'file.random_extension' => 'file.ext'
function getFileNameExt(fileName, ext = 'pdf') {
  return fileName.replace(/\.[0-9a-zA-Z]+$/, `.${ext}`)
}

module.exports = { getSizeInByte, getFileName, getCleanFilename, getFileNameExt
}
```

src/wkhtmltopdf.js

to top

```
const { spawnSync, spawn } = require('child_process');

module.exports = function pdf(templatePath, outputPath, options = {}) {
  const { outline = true } = options;
  const args = [
    '--disable-javascript',
    '--enable-local-file-access',
    '--default-header',
    '--margin-top', '0',
    '--margin-bottom', '0',
    '--allow', './html5bp',
  ];

  if (outline) {
    args.push('--outline');
  }

  args.push(templatePath, outputPath);

  const res = spawnSync('wkhtmltopdf', args);
  if (res.error) {
    console.log('fail to generate pdf using wkhtmltopdf', res.error);
    throw new Error(res.error);
  }
};
```